

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Клиент-серверные приложения и сетевые технологии»

Пояснительная записка к курсовой работе

на тему:

«Разработка клиент-серверного приложения Toy Shelf для учета игрушек»

Выполнил: студент группы БИВТ-24-4

Кадыров Артур Рустамович

Подпись: _____

Проверили: Абросимов Никита Андреевич

Шелудяков Пётр Алексеевич

Москва, 2026

Оглавление

1. Предметная область	2
2. Постановка задачи	3
3. Описание архитектуры	4
4. Описание структуры БД	5
5. Описание серверной части	6
6. Описание клиентской части	7
7. Заключение	8
8. Список литературы	9

1. Предметная область

Предметной областью курсовой работы является разработка персонального клиент-серверного веб-приложения магазина игрушек artlo. Приложение демонстрирует типовой сценарий работы небольшого интернет-магазина: пользователь открывает клиентскую часть в браузере, регистрирует профиль с паролем, входит по email и паролю, просматривает каталог, фильтрует товары по категориям, добавляет игрушки в корзину, оформляет заказ и видит обновление остатков.

Тема «игрушка» выбрана как простая и наглядная предметная область. Игрушка в приложении выступает объектом каталога, который имеет название, категорию, цену, изображение и остаток на складе. Пользователь выступает субъектом системы и содержит идентификатор, имя, адрес электронной почты и необязательный возраст. Такое разделение позволяет показать и предметную часть интерфейса, и обязательную часть задания, связанную с API пользователей.

Проблема предметной области состоит в необходимости централизованно получать и проверять данные, а также ограничивать действия пользователей по категориям доступа. В простом приложении без валидации пользователь может отправить пустое имя, некорректный email или слишком короткий пароль. Без разделения доступа любой посетитель мог бы выполнять административные действия. Поэтому в приложении предусмотрены серверная проверка входных данных, обработка ошибок, вход пользователя и отдельный вход администратора.

Приложение ориентировано на учебную демонстрацию возможностей курса: HTML и CSS применяются в клиентском интерфейсе, LocalStorage используется для сохранения демо-сессии, REST API реализует обмен между клиентом и сервером, а NestJS показывает работу контроллеров, сервисов, провайдеров, декораторов, DTO и фильтров исключений.

2. Постановка задачи

Цель работы — разработать персональное клиент-серверное приложение Toy Shelf, проверить его работоспособность и подготовить пояснительную записку по выполненной работе. Приложение должно демонстрировать основные изученные возможности: HTML, CSS, API, LocalStorage, Git, управление доступом по категориям клиентов, обработку HTTP-запросов и валидацию данных.

В серверной части необходимо реализовать REST API для работы с пользователями. Обязательные endpoint: GET /users — получение списка всех пользователей, GET /users/:id — получение пользователя по идентификатору, POST /users — создание нового пользователя. Каждый пользователь содержит поля id, name, email и age. Поля id, name и email являются обязательными, поле age является необязательным.

Поле email должно проходить проверку формата адреса электронной почты. Для этого в проекте используется библиотека class-validator. Входные данные проверяются через DTO и глобальный ValidationPipe, что позволяет отклонять некорректные запросы до попадания данных в сервисную логику.

По индивидуальной постановке задания данные пользователей должны храниться в памяти и не должны сохраняться между перезапусками приложения. Поэтому в проекте не используется постоянная база данных: массив пользователей хранится внутри UsersService. Это решение соответствует специальному требованию о временном хранении данных. В PDF с общими требованиями к курсовой работе приведены расширенные требования к БД; для данной версии задания они не применяются, поскольку противоречат условию о хранении данных в памяти.

Дополнительно реализованы DELETE /users/:id для удаления пользователя администратором, POST /auth/login для входа пользователя или администратора, GET /toys для вывода каталога игрушек и POST /toys/checkout для оформления заказа. На клиентской стороне реализована корзина, которая сохраняется в LocalStorage. При оформлении заказа сервер уменьшает остатки выбранных товаров в памяти и возвращает обновленный каталог.

3. Описание архитектуры

Toy Shelf построен по клиент-серверной архитектуре. Клиентская часть реализована на React и запускается через Vite. Серверная часть реализована на NestJS и запускается как отдельное HTTP-приложение. Клиент и сервер взаимодействуют по протоколу HTTP, а данные передаются в формате JSON.

Клиентская часть отвечает за отображение начальной страницы входа и регистрации, витрины магазина, корзины, ввод данных, отправку fetch-запросов, сохранение профиля

и корзины в LocalStorage. Серверная часть отвечает за маршрутизацию, валидацию данных, выполнение бизнес-логики, формирование JSON-ответов, обработку исключений, проверку паролей, удаление пользователей, обновление остатков и логирование запросов.

Архитектура сервера разделена на модули. UsersModule отвечает за пользователей, AuthModule оставлен как демонстрационный модуль авторизации, ToysModule отвечает за каталог игрушек и оформление заказа. Такое разделение упрощает сопровождение проекта: каждая предметная область имеет собственный контроллер и сервис.

Контроллеры принимают HTTP-запросы и извлекают из них параметры. Сервисы выполняют прикладную логику и хранят данные. DTO описывают структуру входных данных и правила проверки. Провайдеры NestJS позволяют внедрять сервисы в контроллеры через dependency injection.

В приложении используется CORS, чтобы React-клиент мог обращаться к API NestJS во время локальной разработки. Сервер запускается на порту 3000, клиент Vite обычно запускается на порту 5173.

Компонент	Назначение
React-клиент	Отображение интерфейса, работа с LocalStorage, отправка HTTP-запросов
NestJS-сервер	REST API, валидация, обработка ошибок и логирование
UsersModule	Работа с пользователями
AuthModule	Демонстрационная авторизация и категории доступа
ToysModule	Демонстрационный каталог игрушек

Таблица 1 — Компоненты архитектуры

4. Описание структуры БД

В данной реализации постоянная база данных отсутствует, потому что индивидуальное задание требует хранить данные в памяти и не сохранять их между перезапусками приложения. Поэтому раздел описывает не физическую БД, а структуру in-memory данных, используемых серверной частью.

Основная структура данных — User. Она используется обязательными endpoint /users и хранится в массиве внутри UsersService. При создании пользователя сервер самостоятельно генерирует id с помощью randomUUID. Клиент не передает id в POST-запросе, что предотвращает конфликт идентификаторов.

Сущность User включает поля id, name, email, passwordHash и age. Поле passwordHash используется только внутри сервера и не возвращается клиенту в GET /users. При регистрации пользователь вводит пароль, сервер хеширует его через bcryptjs и

сохраняет только хеш. Перед сохранением email приводится к нижнему регистру, а name очищается от лишних пробелов по краям.

Дополнительная структура Toy используется для отображения темы приложения. Она содержит id, title, category, price, available, stock, rating, description и image. В каталоге используется пять товаров: радиоуправляемая машина Subaru Drift 4WD, робот Vector, кукла Candy, пазл Planet Earth и плюшевый динозавр Rex. Данные игрушек хранятся в памяти и служат для демонстрации карточек товаров, фильтра категорий, корзины и обновления остатков.

При перезапуске backend-приложения все созданные во время работы пользователи и измененные остатки исчезают, а сервисы снова инициализируются начальными демонстрационными данными. Это поведение проверяется вручную: зарегистрировать пользователя, оформить заказ, убедиться в уменьшении остатка, перезапустить сервер и повторно открыть GET /users и GET /toys.

Поле	Тип	Обязательность	Назначение
id	string	да	Уникальный идентификатор пользователя
name	string	да	Имя пользователя
email	string	да	Адрес электронной почты
age	number	нет	Возраст пользователя

Таблица 2 — Структура сущности User

5. Описание серверной части

Серверная часть реализована на NestJS. Входной файл main.ts создает приложение через NestFactory, включает CORS, подключает глобальный ValidationPipe, регистрирует фильтр исключений HttpExceptionHandler и middleware логирования RequestLoggerMiddleware.

AppModule является корневым модулем приложения. Он подключает UsersModule, AuthModule и ToysModule. Модульная структура соответствует подходу NestJS: каждая область содержит собственный контроллер и сервис, а сервисы регистрируются как провайдеры.

UsersController реализует обязательные endpoint задания. Метод findAll обрабатывает GET /users и возвращает массив пользователей без passwordHash. Метод findOne обрабатывает GET /users/:id и передает id в сервис. Метод create обрабатывает POST /users, принимает DTO из тела запроса и создает пользователя. Дополнительно реализован DELETE /users/:id для удаления пользователей администратором через интерфейс.

UserService содержит прикладную логику работы с пользователями. Метод findAll возвращает публичный массив, findOne ищет пользователя по id и выбрасывает NotFoundException при отсутствии записи, create проверяет уникальность email и выбрасывает ConflictException при повторе адреса. После проверки создается объект User с новым id и bcrypt-хешем пароля.

CreateUserDto описывает входные данные POST /users. Для name используются IsString и IsNotEmpty. Для email используются IsEmail и IsNotEmpty. Для password используется MinLength(6). Для age используются IsOptional, Type, IsInt, Min и Max. Благодаря этому сервер отклоняет пустые обязательные поля, некорректный email, слишком короткий пароль, лишние поля и некорректный возраст.

HttpExceptionFilter приводит ошибки к единому виду JSON. Ответ содержит statusCode, timestamp и details. Такой формат удобен для клиента: frontend может получить сообщение ошибки и вывести его пользователю в интерфейсе.

RequestLoggerMiddleware фиксирует метод запроса, URL, HTTP-статус и время выполнения в миллисекундах. Логирование помогает при защите показать, что сервер действительно получает запросы от клиента и корректно отвечает на них.

ToysController содержит GET /toys и POST /toys/checkout. Метод checkout принимает товары из корзины, проверяет существование каждого товара, проверяет положительное количество и достаточный остаток. После успешной проверки сервис уменьшает stock, обновляет available и возвращает сумму заказа вместе с обновленным каталогом.

AuthService выполняет вход пользователей и администратора. Для администратора используется демонстрационная учетная запись admin/admin123. Для обычного пользователя логином является email, а пароль сравнивается с bcrypt-хешем, который хранится в UserService. В ответ клиент получает роль, token и публичные данные пользователя.

Проверка API выполняется через браузер, Postman, Thunder Client или PowerShell. Например, GET http://localhost:3000/users возвращает массив пользователей, POST http://localhost:3000/users создает профиль, GET http://localhost:3000/toys возвращает каталог, а POST http://localhost:3000/toys/checkout уменьшает остатки товаров после оформления заказа.

Endpoint	Метод	Назначение
/users	GET	Получение списка всех пользователей
/users/:id	GET	Получение пользователя по id
/users	POST	Создание пользователя
/auth/login	POST	Демонстрационная авторизация

Endpoint	Метод	Назначение
/toys	GET	Получение каталога игрушек

Таблица 3 — Основные endpoint приложения

6. Описание клиентской части

Клиентская часть реализована на React и Vite. Главный компонент App.jsx содержит состояние текущего профиля, формы входа, формы регистрации, списка пользователей, каталога игрушек, корзины, выбранной категории и сообщения об ошибке. Разметка написана на JSX, а стили вынесены в файл styles.css.

Интерфейс начинается со стартовой страницы, где есть два режима: вход и регистрация. При регистрации пользователь вводит имя, email, пароль и возраст, после чего frontend отправляет POST /users. При входе пользователь вводит email и пароль, а администратор вводит admin и admin123.

Страница магазина содержит заголовок artlo, название «Магазин игрушек», карточку текущего пользователя, статистику по товарам и корзине, фильтр категорий, карточки товаров с фотографиями и корзину с расчетом итоговой суммы. Если выполнен вход администратора, в блоке пользователей появляются кнопки удаления.

Функции loadSession, saveSession и clearSession находятся в storage.js. Они работают с ключом toy-shelf-session. Для корзины используются loadCart, saveCart и clearCart с отдельным ключом toy-shelf-cart. LocalStorage выбран потому, что он изучается в рамках курса и позволяет показать хранение состояния на стороне клиента. При этом чувствительные данные, например реальные пароли или платежные сведения, в LocalStorage не сохраняются.

Функция request из api.js инкапсулирует работу с Fetch API. Она отправляет запрос на сервер, разбирает JSON-ответ, проверяет HTTP-статус и преобразует ошибки в текстовое сообщение. Благодаря этому App.jsx не дублирует обработку ошибок для каждого запроса.

Оформление заказа выполняется из корзины. Клиент отправляет POST /toys/checkout со списком товаров и количеством. После успешного ответа корзина очищается, а каталог обновляется: если товара было 8 и пользователь заказал 1, на карточке остается 7. Так демонстрируется изменение состояния серверной in-memory модели.

CSS-файл задает визуальное оформление приложения: страницу регистрации, сетку каталога, карточки товаров, изображения, фильтр категорий, корзину, кнопки и адаптивное поведение. На экранах небольшой ширины сетка перестраивается в одну колонку.

Для отчета рекомендуется приложить скриншоты: страницу входа и регистрации, регистрацию пользователя с паролем, вход пользователя, вход администратора, удаление пользователя администратором, магазин artlo, карточки пяти товаров с фотографиями, фильтр категорий, добавление товара в корзину, оформление заказа с изменением остатка, ошибку валидации email, ответ GET /users, ответ GET /toys, ответ POST /toys/checkout и лог запросов backend в терминале.

7. Заключение

В ходе курсовой работы разработано клиент-серверное приложение Toy Shelf. Приложение реализует обязательные endpoint для работы с пользователями, хранит данные в памяти, выполняет валидацию email и обязательных полей, обрабатывает ошибки и возвращает JSON-ответы.

Серверная часть демонстрирует изученные возможности NestJS: модули, контроллеры, сервисы, провайдеры, декораторы, DTO, ValidationPipe, фильтр исключений и middleware логирования. Клиентская часть демонстрирует HTML/JSX, CSS, Fetch API, LocalStorage, регистрацию профиля с паролем, вход пользователя, вход администратора, удаление пользователей, витрину товаров, корзину и изменение остатков после заказа.

Поставленная задача выполнена в соответствии с индивидуальной постановкой: данные не сохраняются между перезапусками, API пользователей реализовано, обязательные поля проверяются, email валидируется, а приложение может быть запущено локально и проверено через браузер.

Возможные направления развития проекта: перенос хранения данных из памяти в PostgreSQL, добавление серверных guards для проверки ролей, использование JWT, добавление полноценного CRUD для игрушек и автоматические тесты для контроллеров и сервисов.

8. Список литературы

[1] Официальная документация NestJS. URL: <https://docs.nestjs.com/>

[2] Официальная документация React. URL: <https://react.dev/>

[3] Документация Vite. URL: <https://vite.dev/>

[4] Документация MDN Web Docs по Fetch API. URL: https://developer.mozilla.org/docs/Web/API/Fetch_API

[5] Документация MDN Web Docs по LocalStorage. URL: <https://developer.mozilla.org/docs/Web/API/Window/localStorage>

[6] Документация class-validator. URL: <https://github.com/typestack/class-validator>

[7] Документация bcryptjs. URL: <https://www.npmjs.com/package/bcryptjs>

[8] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления.

[9] Репозиторий исходного кода приложения Toy Shelf. URL: <https://github.com/Artlogg/toy-shelf-coursework>